

Dimensionality Reduction

Sandro Cumani

sandro.cumani@polito.it

Politecnico di Torino

Dimensionality Reduction

Dimensionality reduction techniques compute a mapping from the n -dimensional feature space to a m -dimensional space, with $m \ll n$

- Compress information
- Remove unwanted variability (noise)
- Simplify classification (reduced effects due to high dimensionality, reduced risk of overfitting)
- Data visualization

Dimensionality Reduction

Different goals may require different approaches

- Compress information: we want to retain the *maximum amount of information* for a given output size
- Improve classification: we want to retain *discriminant information*
- Data visualization: we want 2-D or 3-D representations that preserve as much as possible relationships between different samples

Dimensionality Reduction

We will focus on two linear methods:

- Unsupervised: *Principal Component Analysis* (PCA)
- Supervised: *Linear Discriminant Analysis* (LDA)

In both cases, we want to find a subspace of the feature space that preserves most of the “useful” information

Notes on linear algebra

We recall some linear algebra notions for real matrices that will be used in the following

For a quick reference about matrix properties, you can also refer to

Petersen, Pedersen, “The Matrix Cookbook” (available online)

Notes on linear algebra

Let A be a square symmetric $n \times n$ matrix $A \in \mathbb{R}^{n \times n}$

A admits an *eigen-decomposition*

$$A = V \Sigma V^{-1} = V \Sigma V^T$$

- V is an orthogonal $n \times n$ matrix whose columns are the (right) *eigenvectors* of A
- Σ is a diagonal $n \times n$ matrix whose elements are the *eigenvalues* of A

Since V is orthogonal, $V^T V = V V^T = I$ and $V^{-1} = V^T$

Notes on linear algebra

A generic rectangular matrix $A \in \mathbb{R}^{n \times m}$ always admits a *Singular Value Decomposition* (SVD) of the form:

$$A = U \Sigma V^T$$

where

- U is an orthogonal $n \times n$ matrix of eigenvectors of AA^T
- V is an orthogonal $m \times m$ matrix of eigenvectors of $A^T A$
- Σ is a diagonal rectangular matrix containing the *singular values* of A (in decreasing order)

Notes on linear algebra

$$A = U \Sigma V$$

$$\Sigma = \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_m \\ & & & & \mathbf{0} \end{bmatrix}$$

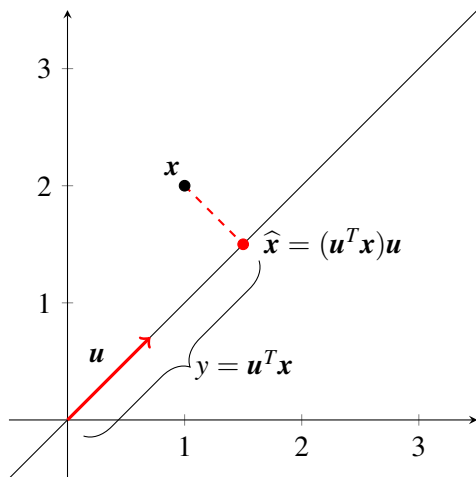
Notes on linear algebra

$$\boxed{A} = \boxed{U} \boxed{\Sigma} \boxed{V}$$

$$\Sigma = \begin{bmatrix} \sigma_1 & & & & \\ & \sigma_2 & & & \\ & & \ddots & & \\ & & & \sigma_m & \\ & & & & \mathbf{0} \end{bmatrix}$$

Notes on linear algebra

Projecting a point over a direction u :



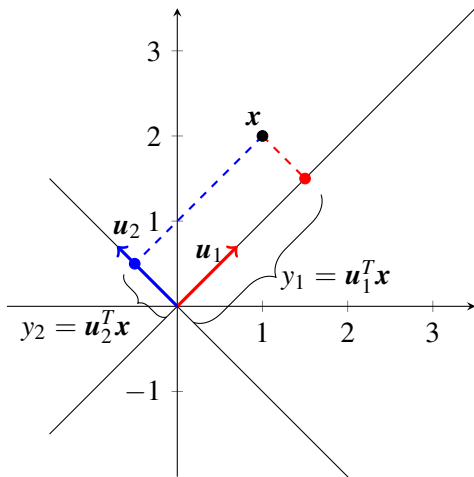
u is a *unit* vector representing a direction

$y = u^T x$ is the *projection* of x over u

$\hat{x} = yu = (u^T x)u$ is the representation of the projected point in the original space (reconstruction)

Notes on linear algebra

Projecting a point in an m -dimensional (sub)space $U = [\mathbf{u}_1 \dots \mathbf{u}_m]$ ($m = 2$ in the example):



The columns of $U = [\mathbf{u}_1, \mathbf{u}_2]$ form a *basis* of $\mathbb{R}^2$

$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} \mathbf{u}_1^T \mathbf{x} \\ \mathbf{u}_2^T \mathbf{x} \end{bmatrix} = U^T \mathbf{x}$ is the *projection* of $\mathbf{x}$ over the column space of U

The representation (reconstruction) of $\mathbf{y}$ in the original space can be obtained as

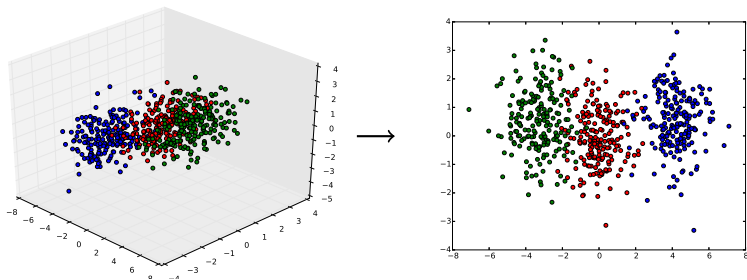
$$\hat{\mathbf{x}} = y_1 \mathbf{u}_1 + y_2 \mathbf{u}_2 = U \mathbf{y} = U U^T \mathbf{x}$$

In this example, since U is full rank, $\hat{\mathbf{x}} = \mathbf{x}$

Principal Component Analysis

We are given a zero-mean dataset $X = \{\mathbf{x}_1, \dots, \mathbf{x}_K\}$, with $\mathbf{x}_i \in \mathbb{R}^n$

We want to find the subspace of $\mathbb{R}^n$ that allows preserving *most* of the information



Principal Component Analysis

A subspace can be represented as a matrix $\mathbf{P} \in \mathbb{R}^{n \times m}$ whose columns are *orthonormal*

The columns of $\mathbf{P}$ form a basis of a subspace of $\mathbb{R}^n$ with dimension m

The projection of $\mathbf{x}$ over the subspace is given by $\mathbf{y} = \mathbf{P}^T \mathbf{x}$:

$$\mathbf{y} = \begin{bmatrix} \mathbf{p}_1^T \mathbf{x} \\ \mathbf{p}_2^T \mathbf{x} \\ \vdots \\ \mathbf{p}_m^T \mathbf{x} \end{bmatrix}$$

where $\mathbf{p}_1 \dots \mathbf{p}_m$ are the columns of $\mathbf{P}$

We can compute the coordinates of the projected point $\mathbf{y}$ in the original space as $\hat{\mathbf{x}} = \mathbf{P}\mathbf{y}$

Principal Component Analysis

We need to define a criterion for estimating $\mathbf{P}$

A reasonable criterion may be the minimization of the *average reconstruction error* (K is the number of samples)

$$\frac{1}{K} \sum_{i=1}^K \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \frac{1}{K} \sum_{i=1}^K \|\mathbf{x}_i - \mathbf{P}\mathbf{y}_i\|^2 = \frac{1}{K} \sum_{i=1}^K \|\mathbf{x}_i - \mathbf{P}\mathbf{P}^T \mathbf{x}_i\|^2$$

We therefore want to solve

$$\mathbf{P}^* = \arg \min_{\mathbf{P}} \frac{1}{K} \sum_{i=1}^K \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \arg \min_{\mathbf{P}} \frac{1}{K} \sum_{i=1}^K \|\mathbf{x}_i - \mathbf{P}\mathbf{P}^T \mathbf{x}_i\|^2$$

Principal Component Analysis

We can rewrite the objective function as

$$\begin{aligned}\mathcal{L}(\mathbf{P}) &= \frac{1}{K} \sum_{i=1}^K \|\mathbf{x}_i - \mathbf{P}\mathbf{P}^T \mathbf{x}_i\|^2 = \frac{1}{K} \sum_{i=1}^K (\mathbf{x}_i^T \mathbf{x}_i - 2\mathbf{x}_i^T \mathbf{P}\mathbf{P}^T \mathbf{x}_i + \mathbf{x}_i^T \mathbf{P}\mathbf{P}^T \mathbf{P}\mathbf{P}^T \mathbf{x}_i) \\ &= \frac{1}{K} \sum_{i=1}^K (\mathbf{x}_i^T \mathbf{x}_i - 2\mathbf{x}_i^T \mathbf{P}\mathbf{P}^T \mathbf{x}_i + \mathbf{x}_i^T \mathbf{P}\mathbf{P}^T \mathbf{x}_i) \\ &= \frac{1}{K} \sum_{i=1}^K (\mathbf{x}_i^T \mathbf{x}_i - \text{Tr}(\mathbf{P}^T \mathbf{x}_i \mathbf{x}_i^T \mathbf{P}))\end{aligned}$$

where Tr represents the trace of a matrix, and we used the fact that $\mathbf{P}^T \mathbf{P} = \mathbf{I}$

Here we used the fact that, for any two vectors $\mathbf{v}$ and $\mathbf{w}$

$$\mathbf{v}^T \mathbf{w} = \text{Tr}(\mathbf{v}^T \mathbf{w})$$

and the property that

$$\text{Tr}(\mathbf{A}\mathbf{B}) = \text{Tr}(\mathbf{B}\mathbf{A})$$

Principal Component Analysis

Since Tr is a linear operator, minimizing $\mathcal{L}$ is equivalent to maximizing

$$\hat{\mathcal{L}}(\mathbf{P}) = \text{Tr} \left(\mathbf{P}^T \left[\frac{1}{K} \sum_{i=1}^K \mathbf{x}_i \mathbf{x}_i^T \right] \mathbf{P} \right)$$

We require that $\mathbf{P}$ is an orthogonal matrix

It can be shown that the optimal solution is then given by the matrix $\mathbf{P}$ whose columns are the m *eigenvectors* of $\frac{1}{K} \sum_{i=1}^K \mathbf{x}_i \mathbf{x}_i^T$ corresponding to the m *largest eigenvalues*

Principal Component Analysis

Let

$$\frac{1}{K} \sum_{i=1}^K \mathbf{x}_i \mathbf{x}_i^T = \mathbf{U} \mathbf{\Sigma} \mathbf{U}^T$$

be an eigen-decomposition of the matrix, with $\mathbf{\Sigma}$ containing the eigenvalues in *descending* order

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_n \end{bmatrix}, \quad \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n$$

and

$$\mathbf{U} = [\mathbf{u}_1 \dots \mathbf{u}_m, \mathbf{u}_{m+1} \dots \mathbf{u}_n]$$

Then

$$\mathbf{P}^* = [\mathbf{u}_1 \dots \mathbf{u}_m]$$

i.e., $\mathbf{P}^*$ corresponds to the first m columns of $\mathbf{U}$

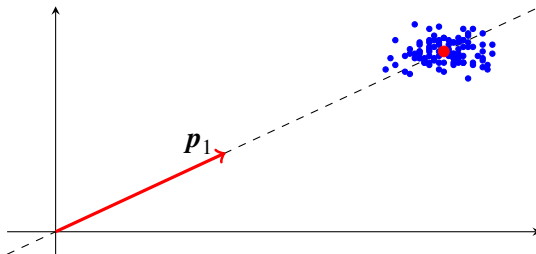
Principal Component Analysis

What happens if the dataset is not zero-mean?

P represents a subspace, whose axes pass through the origin

If the dataset is far from the origin, the first PCA direction will approximately connect the origin and the dataset mean

This direction (in most cases) is not very interesting



Principal Component Analysis

We can recast the problem as looking for the projection surface¹ (line, plane, ...) that minimizes the reconstruction error

In practice, we can cast the problem as jointly looking for a dataset shift m and a subspace U over which we can project the shifted data

An optimal solution for the shift is given by the dataset mean

In practice, we remove the dataset mean before computing PCA

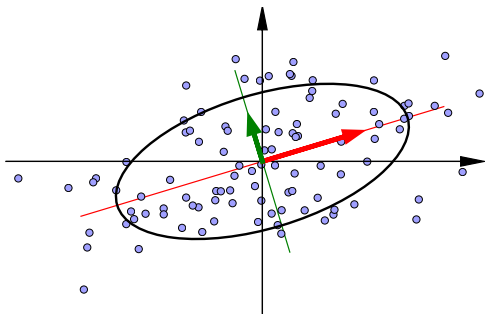
If the dataset mean is $\bar{x}$, the PCA subspace is computed from the eigenvectors of the empirical covariance matrix

$$\frac{1}{K} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$$

¹Not necessarily a subspace

Principal Component Analysis

PCA can be interpreted as the linear mapping that preserves the directions with highest variance



The axes of the ellipse correspond to the principal directions

Principal Component Analysis

Consider the covariance matrix

$$\mathbf{C} = \frac{1}{K} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$$

We can compute the eigen-decomposition of $\mathbf{C}$

$$\mathbf{C} = \mathbf{U}\mathbf{\Sigma}\mathbf{U}^T$$

$\mathbf{U}$: Eigenvectors matrix

$\mathbf{\Sigma}$: Diagonal eigenvalues matrix

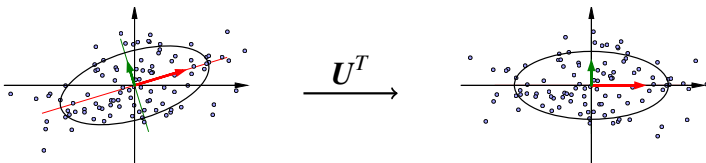
Remember that $\mathbf{U}^{-1} = \mathbf{U}^T$, thus $\mathbf{U}^T \mathbf{U} = \mathbf{I}$

Principal Component Analysis

Projecting our data centered points over U^T we obtain

$$C' = \frac{1}{K} \sum_i U^T (x_i - \bar{x})(x_i - \bar{x})^T U = U^T (U \Sigma U^T) U = \Sigma$$

Projection over U^T transforms our data so that the different directions are *uncorrelated*



To keep only the m directions with highest variance we keep only the first m transformed directions

Principal Component Analysis

Usually PCA is applied directly to *centered* data

- Compute sample mean $\bar{x}$
- Center data: $z_i = x_i - \bar{x}$
- Compute the sample covariance matrix

$$C = \frac{1}{K} \sum_i (x_i - \bar{x})(x_i - \bar{x})^T = \frac{1}{k} \sum_i z_i z_i^T$$

- Compute the eigen-decomposition of $C = U \Sigma U^T$
- Project the data in the subspace spanned by the m columns of U corresponding to the m highest eigenvalues (matrix P):
 $y_i = P^T z_i = P^T (x_i - \bar{x})$
- Reconstruction requires inverting the process: $\hat{x}_i = \bar{P} y_i + \bar{x}$

Principal Component Analysis

Selection of optimal m can be done by cross-validation using a *validation* set

We can also select m as to retain a given percentage t (e.g. 95%) of the variance of the data

Remember that each eigenvalue corresponds to the variance along the corresponding axis

We choose m as the lowest number for which the sum of the first m eigenvalues divided by the sum of all eigenvalues is larger than t

$$\min_m m \quad \text{s.t.} \quad \frac{\sum_{i=1}^m \sigma_i}{\sum_{i=1}^n \sigma_i} \geq t$$

where σ_i is the i -th largest eigenvalue (diagonal element of Σ)

Principal Component Analysis

Computing the sample covariance can be difficult when the feature space is very large

Different solutions

- Truncated Singular Value Decomposition
- Probabilistic PCA

For smaller datasets the standard approach is sufficient

MNIST Dataset

To compare different methods we consider the MNIST handwritten digits dataset

Task: classification of handwritten digits

- Classes are digits $0, 1, \dots, 9$
- Images have already been normalized (centered and scaled to a square 28×28 shape)
- Gray-scale images (originally binary)
- 60 000 training images (6000 for each digit)
- 10 000 test images

MNIST Dataset

Train samples:



Test samples:

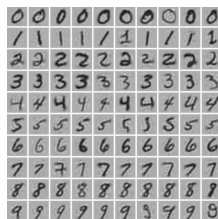


In general, we will consider both multi-class and pair-wise binary classification tasks

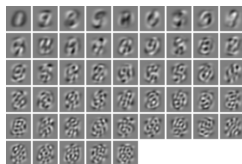
In the following we show the results of PCA applied to the images

Principal Component Analysis

50 dimensional PCA



U :



Principal Component Analysis

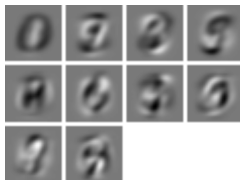
10 dimensional PCA

0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9



0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9

U :



Dimensionality Reduction

To analyze the results of PCA as pre-processing for classification we consider a very simple classifier based on Euclidean distance

For each class, we compute the *mean* vector $\boldsymbol{\mu}_c = \frac{1}{n_c} \sum_{i=1}^{n_c} \mathbf{x}_{c,i}$

For a test sample $\mathbf{x}_t$ we predict its label as the label of the class whose mean is closest to the test sample itself:

$$c_t = \arg \min_c \|(\mathbf{x}_t - \boldsymbol{\mu}_c)\|^2$$

Dimensionality Reduction

We measure the error rate as the number of incorrect classified samples over the total number of samples (we will see better measures in the next weeks)

	m^2			
w/o PCA	100	50	9	5
18.0%	18.1%	18.2%	25.5%	35.9%

Most of the dimensions can be safely removed — the results with 50 and 100 dimensions are very close to those of the full image

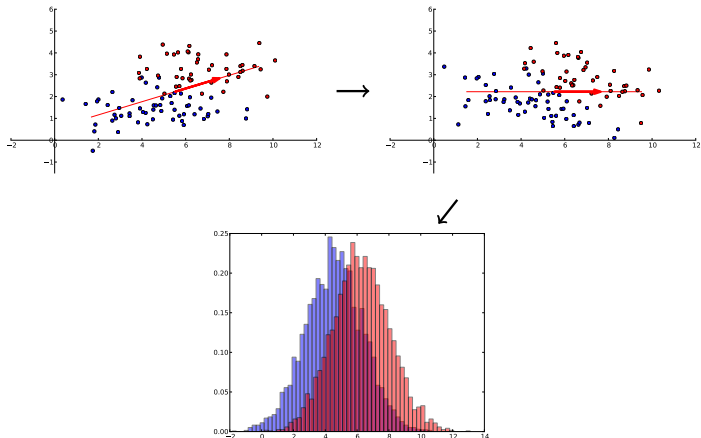
If we remove too much information, classification accuracy drops significantly

For the Euclidean classifier on MNIST, PCA is not helpful in removing unwanted variability

²Remember that the optimal dimensionality m should be selected according to results on a validation set!

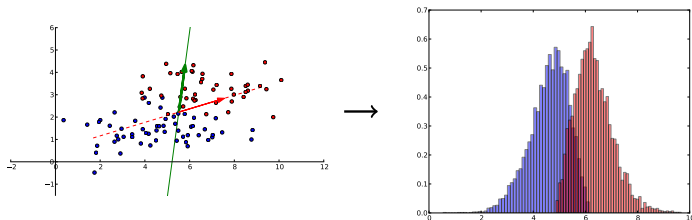
Linear Discriminant Analysis

PCA is unsupervised: no guarantee of obtaining *discriminant* directions



Linear Discriminant Analysis

We want a transformation that allows us to better separate the classes



We represent a direction as a unit vector $\mathbf{w}$

The projected point is $y = \mathbf{w}^T \mathbf{x}$ (a scalar)

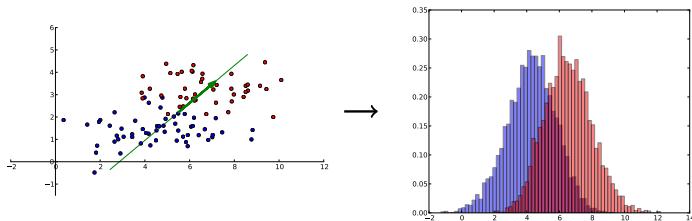
Red line: PCA

Green line: Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis

We could consider the line connecting the class means

$$w \propto \mu_2 - \mu_1$$



Problem: if the data points of each class are scattered along the same directions of the class mean, we still cannot properly separate the classes

Linear Discriminant Analysis

Fisher Linear Discriminant Analysis: find a direction that has a large separation between the classes and small spread inside each class

We measure spread in terms of class *covariance*

LDA: maximize the *between-class* variability over *within-class* variability ratio for the transformed samples:

$$\max_w \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

Linear Discriminant Analysis

The between and within class variability matrices are defined as

$$S_B \triangleq \frac{1}{N} \sum_{c=1}^K n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu}) (\boldsymbol{\mu}_c - \boldsymbol{\mu})^T$$
$$S_W \triangleq \frac{1}{N} \sum_{c=1}^K \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c) (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T$$

where $\mathbf{x}_{c,i}$ is the i -th sample of class c , n_c is the number of samples of class c , K is the total number of classes, N is the total number of samples $N = \sum_{c=1}^K n_c$, and

- $\boldsymbol{\mu}$ is the dataset mean $\boldsymbol{\mu} = \frac{1}{N} \sum_{c=1}^K \sum_i \mathbf{x}_{c,i}$
- $\boldsymbol{\mu}_c$ is the mean of class c $\boldsymbol{\mu}_c = \frac{1}{n_c} \sum_{i=1}^{n_c} \mathbf{x}_{c,i}$

Linear Discriminant Analysis

The between class covariance matrix can be interpreted as a covariance matrix for the class means, where each class is weighted by the corresponding sample size n_c

The within class covariance matrix can be seen as a (also weighted) average of the covariance matrix of each class

We can observe that

$$\mathbf{S}_B + \mathbf{S}_W = \frac{1}{N} \sum_{c=1}^K \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu})(\mathbf{x}_{c,i} - \boldsymbol{\mu})^T$$

i.e., the covariance matrix of the dataset as a whole

Linear Discriminant Analysis

Since we are looking for a discriminant direction $\mathbf{w}$, we now consider the between and within class variance of the projected samples $\mathbf{w}^T \mathbf{x}$

The global mean and class means in the direction $\mathbf{w}$ are simply

$$m = \mathbf{w}^T \boldsymbol{\mu}, \quad m_c = \mathbf{w}^T \boldsymbol{\mu}_c$$

The between and within class variance over the direction $\mathbf{w}$ can also be computed as:

$$s_B = \frac{1}{N} \sum_{c=1}^K n_c (\mathbf{w}^T \boldsymbol{\mu}_c - \mathbf{w}^T \boldsymbol{\mu}) (\mathbf{w}^T \boldsymbol{\mu}_c - \mathbf{w}^T \boldsymbol{\mu})^T = \mathbf{w}^T \mathbf{S}_B \mathbf{w}$$

$$s_W = \frac{1}{N} \sum_{c=1}^K \sum_{i=1}^{n_c} n_c (\mathbf{w}^T \mathbf{x}_{c,i} - \mathbf{w}^T \boldsymbol{\mu}_c) (\mathbf{w}^T \mathbf{x}_{c,i} - \mathbf{w}^T \boldsymbol{\mu}_c)^T = \mathbf{w}^T \mathbf{S}_W \mathbf{w}$$

Linear Discriminant Analysis

Fisher discriminant analysis defines as criterion of optimality the maximization of the *ratio of between and within class variance* for the projected points

We assume that S_W is *full rank*, thus the *objective function* is

$$\mathcal{L}(\mathbf{w}) = \frac{s_B}{s_W} = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

Note that the criterion does not depend on the scale of $\mathbf{w}$, i.e. if $\mathbf{w}$ is a maximizer of $\mathcal{L}$, then $\alpha \mathbf{w}$ is also a maximizer of $\mathcal{L}$. We can therefore select a maximizer with *unit norm*.

Linear Discriminant Analysis

We can find an optimum by solving $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = 0$, where $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$ is the *gradient* of $\mathcal{L}$:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = 2 \frac{\mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}} - 2 \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w} \mathbf{S}_W \mathbf{w}}{(\mathbf{w}^T \mathbf{S}_W \mathbf{w})^2} = 0$$

The optimum is obtained for

$$(\mathbf{w}^T \mathbf{S}_W \mathbf{w}) \mathbf{S}_B \mathbf{w} = (\mathbf{w}^T \mathbf{S}_B \mathbf{w}) \mathbf{S}_W \mathbf{w}$$

i.e.

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda(\mathbf{w}) \mathbf{w}$$

where

$$\lambda(\mathbf{w}) = \mathcal{L}(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

Linear Discriminant Analysis

We can observe that

- The optimal solution is an eigenvector of $S_W^{-1}S_B$
- The eigenvalue corresponding to solution $\mathbf{w}$ is $\lambda(\mathbf{w}) = \mathcal{L}(\mathbf{w})$, i.e. the value of the ratio we want to maximize

The maximum of $\mathcal{L}$ is thus the *eigenvector* of $S_W^{-1}S_B$ corresponding to the *largest eigenvalue*

Linear Discriminant Analysis

The method was originally introduced to solve binary problems

Indeed, once we have estimated $\mathbf{w}$, we can project our test samples over $\mathbf{w}$, and assign the class according to whether the projected value (score) is larger or lower than a given threshold³:

$$C(\mathbf{x}_t) = \begin{cases} C_1 & \text{if } \mathbf{w}^T \mathbf{x}_t \geq t \\ C_2 & \text{if } \mathbf{w}^T \mathbf{x}_t < t \end{cases}$$

³How to select a good threshold will be a topic for next classes

Linear Discriminant Analysis

For the binary problem, we can express the *between-class* covariance matrix as

$$\mathbf{S}_B = k(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T$$

where k is a constant that depends on the number of samples of each class but is irrelevant to find the optimal direction.

In this case, we can observe that $\text{rank}(\mathbf{S}_B) = 1$, thus $\mathbf{S}_W^{-1}\mathbf{S}_B$ has a single non-zero eigenvalue

It can be verified that the eigenvector of $\mathbf{S}_W^{-1}\mathbf{S}_B$ associated to the non-zero eigenvalue is

$$\mathbf{w} \propto \mathbf{S}_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$$

Linear Discriminant Analysis

Despite being originally introduced to solve binary classification problems, LDA has found large success as a dimensionality reduction technique

In this case, we are interested in looking for the m most discriminant directions

We represent these directions as a matrix $\mathbf{W}$, whose columns contain the directions we want to find

Notice that we do not require that $\mathbf{W}$ is orthogonal

If we want, we can nevertheless find a basis for the subspace spanned by the columns of $\mathbf{W}$

Linear Discriminant Analysis

The projected points are computed as $\hat{\mathbf{x}} = \mathbf{W}^T \mathbf{x}$

We can express the projected between and within class covariance matrices as

$$\begin{aligned}\widehat{\mathbf{S}}_B &= \mathbf{W}^T \mathbf{S}_B \mathbf{W} \\ \widehat{\mathbf{S}}_W &= \mathbf{W}^T \mathbf{S}_W \mathbf{W}\end{aligned}$$

Different criteria can be used to generalize the 1-dimensional case

A common one looks for the maximizer of

$$\mathcal{L} = \text{Tr} \left(\widehat{\mathbf{S}}_W^{-1} \widehat{\mathbf{S}}_B \right)$$

Linear Discriminant Analysis

It can be shown that the solution is given by the m (right) eigenvectors corresponding to the m largest eigenvalues of $S_W^{-1}S_B$

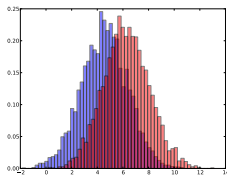
We can also compute the solution by solving the *generalized eigenvalue* problem $S_B\mathbf{w} = \lambda S_W\mathbf{w}$

Notice that, from the definition of S_B , the number of non-zero eigenvalues is at most $C - 1$

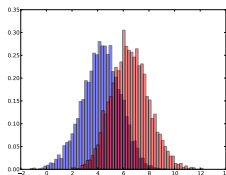
Therefore, LDA allows estimating *at most* $C - 1$ *directions*

Linear Discriminant Analysis

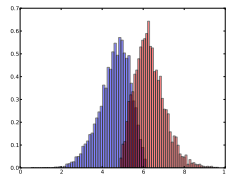
PCA



Eigenvectors of S_B



LDA



Linear Discriminant Analysis

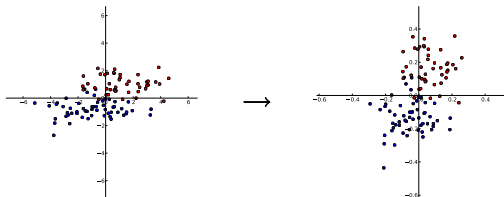
A way to solve the generalized eigenvalue problem, and thus find the LDA matrix W , consists in the joint diagonalization of S_W and S_B that makes S_W become the identity matrix and S_B become a diagonal matrix

Remember that applying the linear transformation $\tilde{x} = Ax$ to our dataset, covariance matrices transform as $\tilde{\Sigma} = A\Sigma A^T$

Linear Discriminant Analysis

Whiten S_W

- Compute the eigen-value decomposition $S_W = U_W \Sigma_W U_W^T$
- Apply the whitening transformation⁴ described by
$$P_W = U_W \Sigma_W^{-\frac{1}{2}} U_W^T$$
- $S_W \rightarrow I, S_B \rightarrow P S_B P^T$

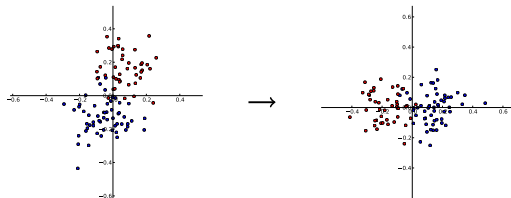


⁴There are different methods to compute the whitening transformation. Indeed, matrix P is defined up to a unitary transformation.

Linear Discriminant Analysis

Diagonalize the transformed S_B

- Compute the eigenvalue decomposition $PS_B P^T = U_B \Sigma_B U_B^T$
- Diagonalize by projecting over U_B^T
- $S_W \rightarrow I, S_B \rightarrow \Sigma_B$



The first m directions of the transformed samples correspond to the LDA subspace. This method will be further discussed in Laboratory 3.

Linear Discriminant Analysis

LDA assumes Gaussian-distributed noise

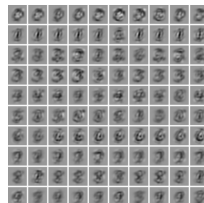
When the number of directions is large S_W can be singular or close to singular

It is often helpful to pre-process our data using PCA before applying LDA

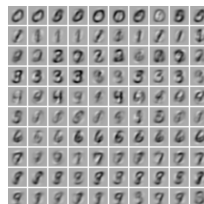
Linear Discriminant Analysis

In MNIST we have 9 classes, so we can have at most 9 directions

0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9



PCA+LDA



PCA

Dimensionality Reduction

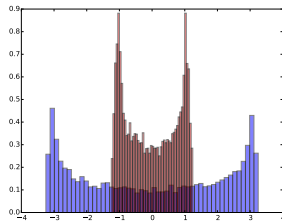
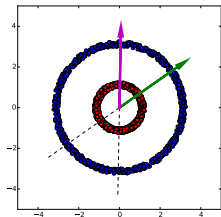
MNIST — Error rates for euclidean distance classifier⁵

m	PCA	PCA+LDA
100	18.1%	—
50	18.2%	—
9	25.5%	12.2%
5	35.9%	17.9%

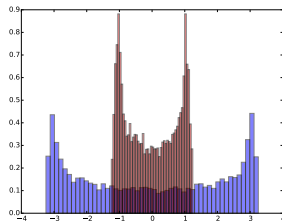
⁵Remember that the method and dimensionality m should be selected according to results on the validation set!

Non-linear Dimensionality Reduction

Linear transformations are not always suited for our data



PCA



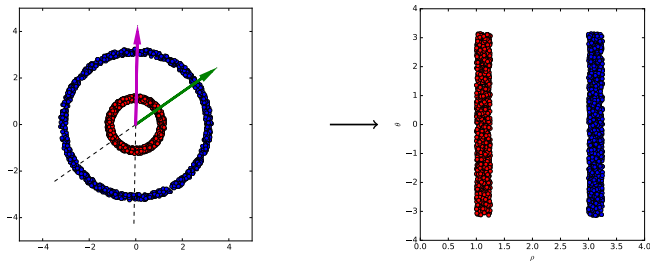
LDA

Non-linear Dimensionality Reduction

We can transform the features so that linear methods are suited for the transformed data

For example, we can represent the 2-dimensional data in the figure through polar coordinates :

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \longrightarrow \mathbf{y} = f(\mathbf{x}) \begin{bmatrix} \rho(\mathbf{x}) \\ \theta(\mathbf{x}) \end{bmatrix}$$



Alternative non-linear methods will be presented in other courses